



# Authorisation & Authentication

COMS 4156W Advanced Software Engineering  
Fall 2024

Is Authorisation == Authentication?

The background of the slide features a series of dark grey, three-dimensional rectangular planes that recede into the distance, creating a sense of depth. A bright green parallelogram is positioned on one of the upper planes, and a bright blue parallelogram is on a lower plane, both adding a modern, geometric aesthetic to the design.

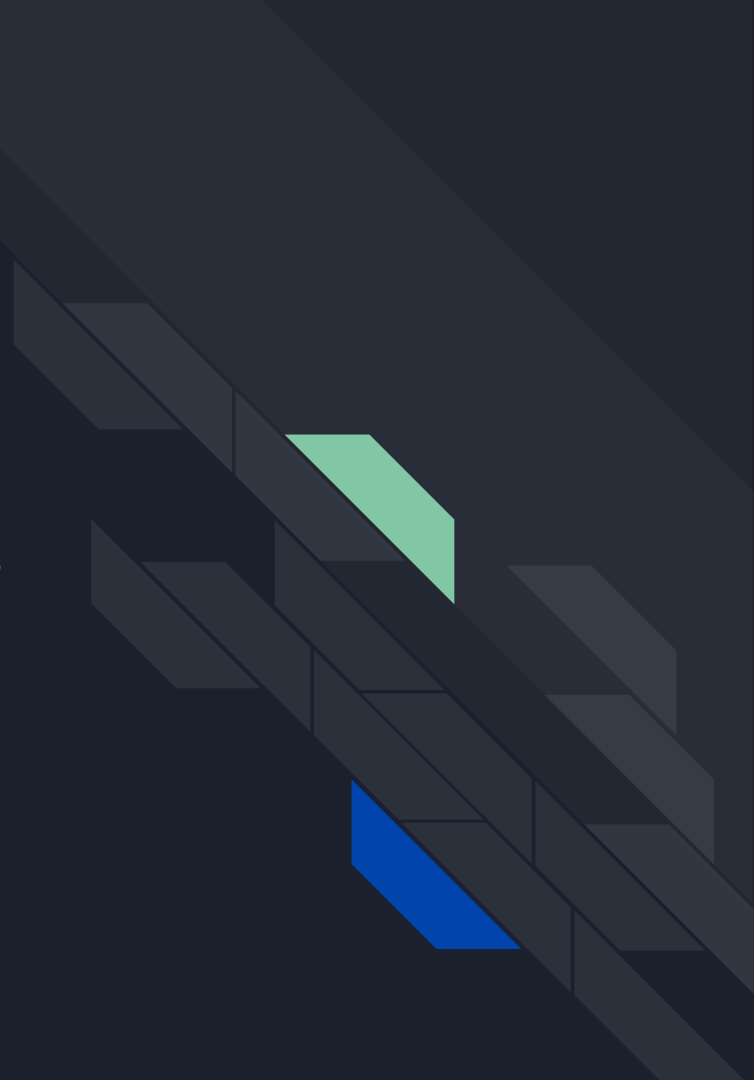
Often confused, Authentication is not same as Authorization.

An easy way to remember what is what->

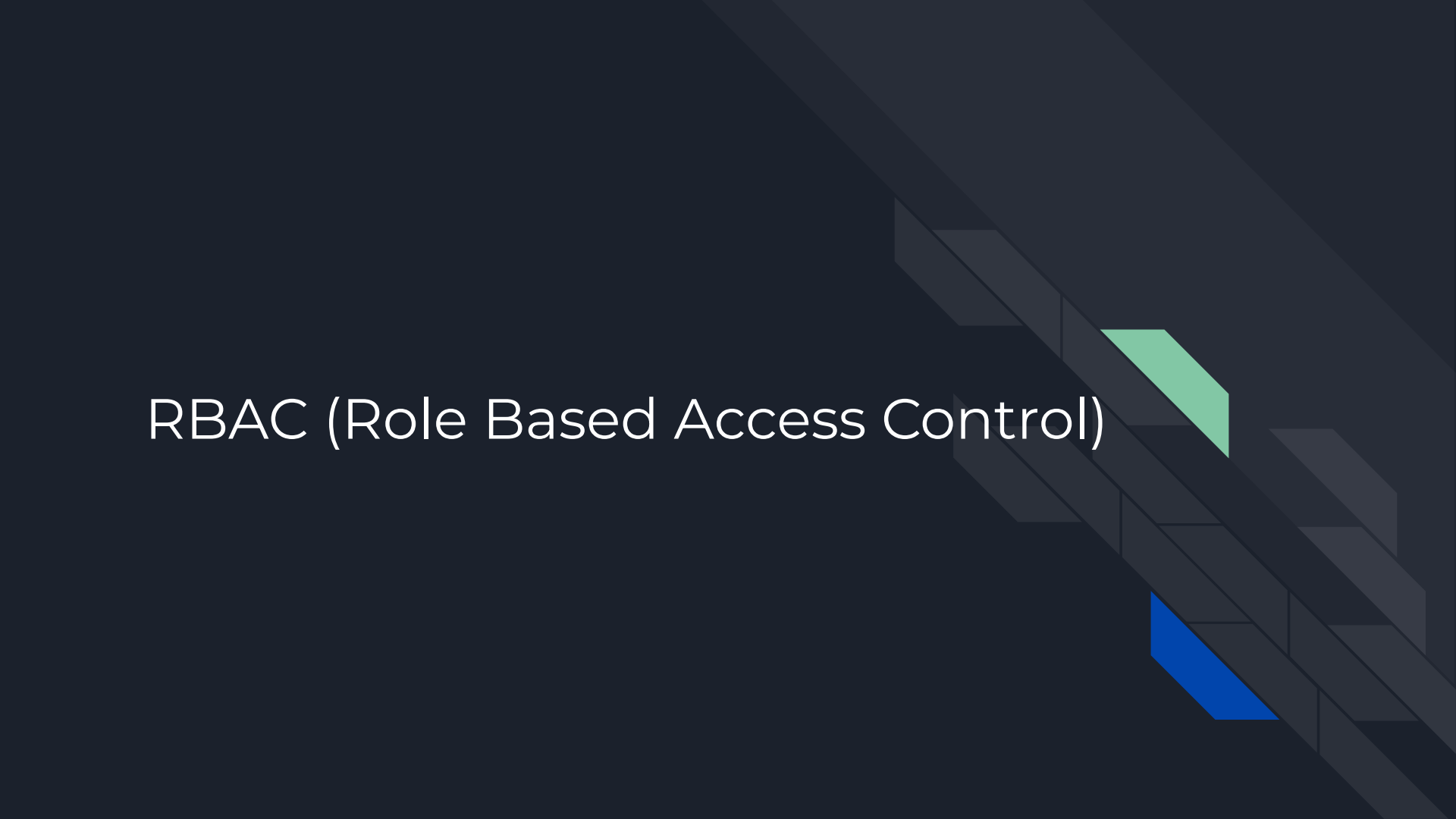
Who you are? → Authentication

What you can do? → Authorization

# Authorization Approaches



# RBAC (Role Based Access Control)

The background features a series of dark gray, three-dimensional rectangular planes that recede into the distance, creating a sense of depth. A light green parallelogram is positioned on one of the upper planes, and a blue parallelogram is on a lower plane, both adding a pop of color to the monochromatic scheme.



# RBAC

1. RBAC is generally too coarse.
2. If you make RBAC fine grained, you end up with Role explosion.
3. Not just that, if you plan to embed Role info in the Access Token, you end up with Token bloat.

Not good! Works for really simple systems where clear cut roles and their functions are defined.

# ABAC (Attribute based Access Control)

The background of the slide features a series of dark gray, three-dimensional rectangular planes that recede into the distance, creating a sense of depth. A bright green parallelogram is positioned in the middle ground, and a blue parallelogram is located further back and lower down, both adding color and geometric interest to the composition.



# ABAC

You define Access Policies per entity that can be accessed.

Every system and user attribute is fair game in the access policy.

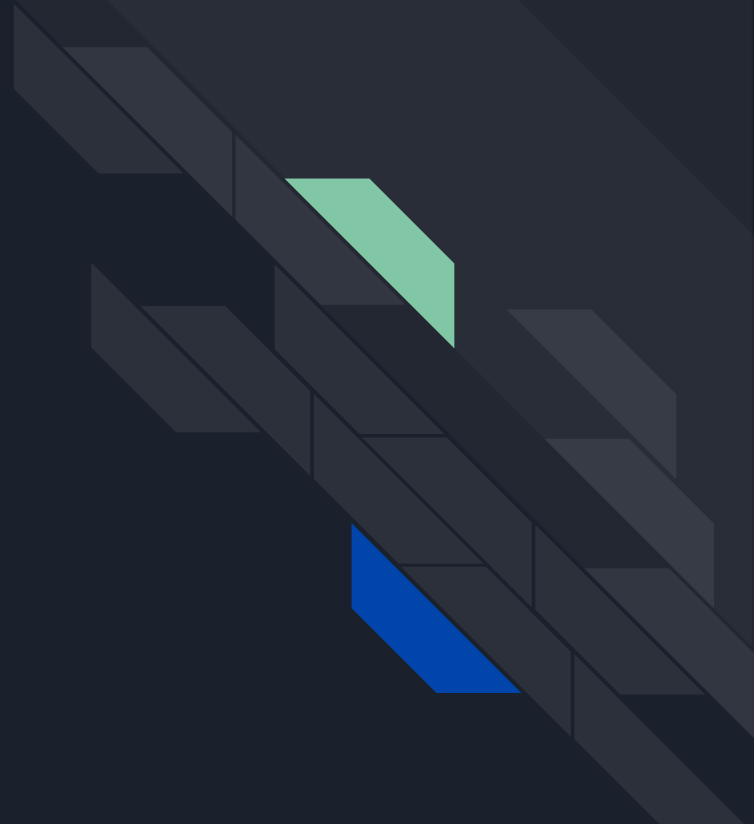
Generally access control is delegated to individual systems to implement.

The auth server would collect all the attributes for the request, send it to the access policy control system of the service, and then allow/not allow the user to access the resource.

Also you would need to define the rules for all the resources, entities, and entities where you need to provide access.

Problem comes in cases where one system can bottleneck the whole authorization process.

Google Zanzibar





# Zanzibar

From the Zanzibar paper →

“Zanzibar provides a uniform data model and configuration language for expressing a wide range of access control policies from hundreds of client services at Google, including Calendar, Cloud, Drive, Maps, Photos, and YouTube.”

# ReBAC (Relationship based Access Control)

The background of the slide features a series of dark gray, three-dimensional rectangular planes that recede into the distance, creating a sense of depth. A bright green parallelogram is positioned in the middle ground, and a blue parallelogram is located further back and lower down, both appearing to be part of the geometric structure.

# ReBAC

How does Zanzibar work?

Namespace: Videos

| Object  | Relation | Userset          |
|---------|----------|------------------|
| Video X | Viewer   | User A           |
| Video Y | Viewer   | All user         |
| Video X | Viewer   | (Group1, Member) |

Namespace: Groups

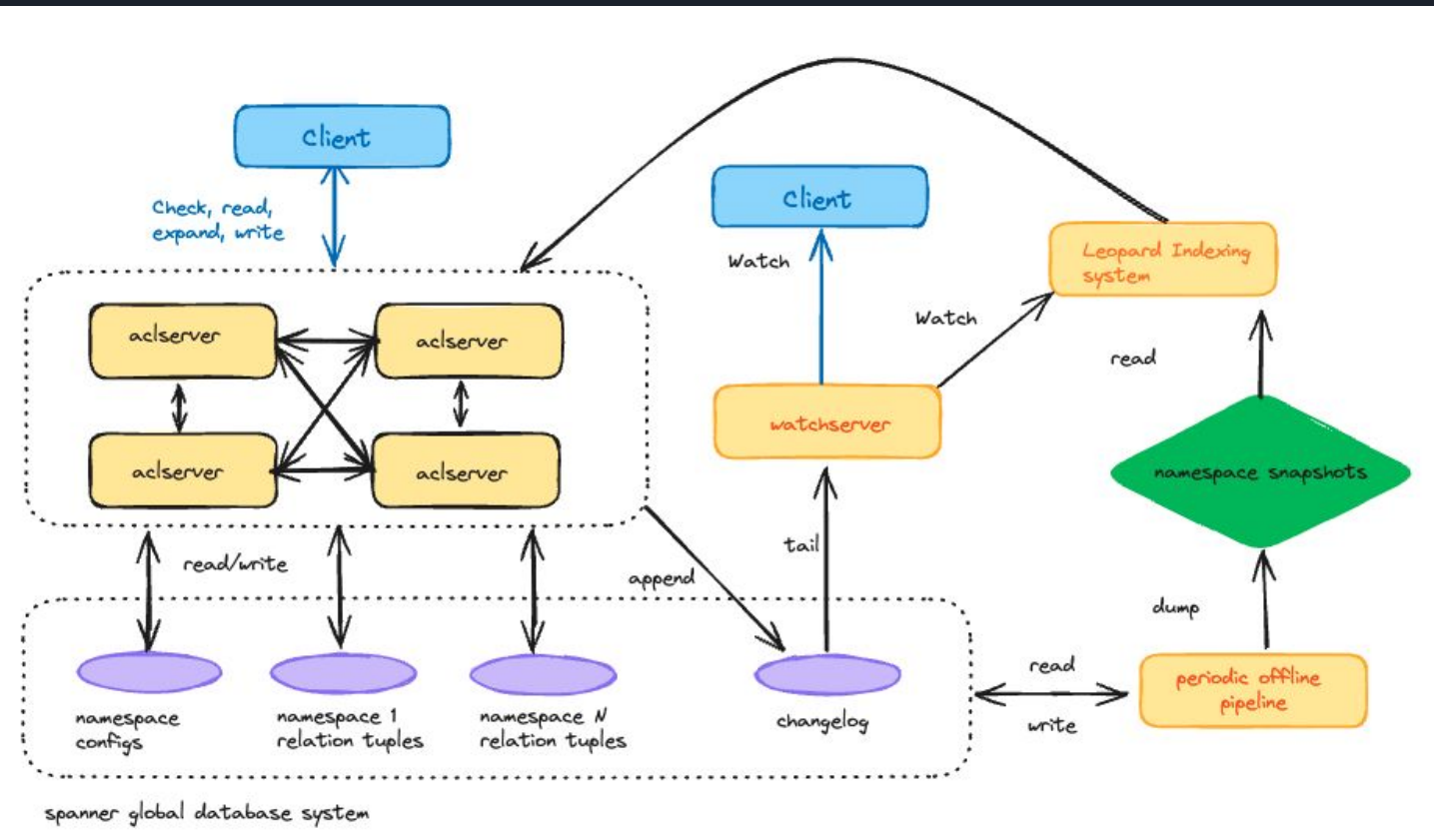
| Object  | Relation | Userset |
|---------|----------|---------|
| Group 1 | Member   | User C  |
| Group 1 | Member   | User D  |
| Group 2 | Member   | User B  |



## Zanzibar features:

- Uses relation tuples to represent access control lists (ACLs) and group memberships
- Supports a flexible configuration language for clients to define relations and access policies
- Allows complex nested group structures and inheritance of permissions.
- Provides external consistency to respect causal ordering of ACL updates
- Stores data in Spanner, a globally distributed database, for consistency and replication
- Zanzibar also supports reverse indexing (also known as data filtering). This means that after assigning a user many individual permissions, you can also ask, "what resources does this user have access to?" This is a common authorization request (e.g., for list endpoints). It's also useful for maintaining and debugging access controls.

# Zanzibar Architecture





# How to add authorization to your application?

Don't reinvent the wheel, use:

1. Casbin
2. Keycloak
3. Open Policy Agent
4. IAM (for AWS resource access control)